
Spring Boot

Principais anotações e conceitos fundamentais



Prof. Dr. João Choma

UEM

github.com/JoaoChoma

Visão geral



Ao final da aula, você deverá ser capaz de:

- Identificar as principais anotações de uma API Spring Boot.
- Explicar Inversão de Controle, Injeção de Dependência e Beans.
- Organizar controller, service e repository em camadas.
- Mapear entidades e validar dados com APIs Jakarta.
- Centralizar o tratamento de erros HTTP.





Combina configuração, autoconfiguração e busca de componentes. Geralmente fica na classe principal.

```
@SpringBootApplication
public class MinhaApiApplication {
    public static void main(String[] args) {
        SpringApplication.run(MinhaApiApplication.class, args);
    }
}
```

Organização

Coloque essa classe em um pacote raiz para que a busca encontre os componentes dos subpacotes.

Camada Web



Marca uma classe como componente Web cujos métodos escrevem dados diretamente na resposta HTTP, normalmente em JSON.

```
@RestController
@RequestMapping("/produtos")
public class ProdutoController {

    @GetMapping
    public List<Produto> listar() {
        return List.of();
    }
}
```



Anotação	Operação comum	Semântica
@GetMapping	Consultar	Leitura
@PostMapping	Criar	Inclusão
@PutMapping	Atualizar	Substituição
@DeleteMapping	Excluir	Remoção

São atalhos especializados de `@RequestMapping` para cada verbo HTTP.



```
@GetMapping("/{id}")
public Produto buscar(@PathVariable Long id) { ... }

@PostMapping
public Produto criar(@RequestBody Produto produto) { ... }

@PutMapping("/{id}")
public Produto atualizar(@PathVariable Long id,
                        @RequestBody Produto produto) { ... }

@DeleteMapping("/{id}")
public void excluir(@PathVariable Long id) { ... }
```



@PathVariable

Extraí um valor variável da URL.

`/produtos/42` $\Rightarrow$ `id = 42`

@RequestBody

Converte o corpo da requisição, como JSON, em um objeto Java.

A URL identifica o recurso; o corpo transporta a representação enviada pelo cliente.

IoC, DI e Beans



Sem IoC

A própria classe instancia e controla suas dependências.

Com IoC

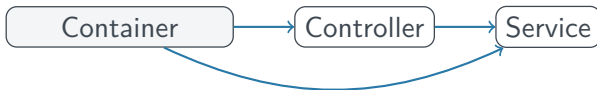
O container Spring cria, configura e conecta os objetos da aplicação.

O controle da criação dos objetos é **invertido**: sai da classe e vai para o container.



Bean objeto criado, configurado e gerenciado pelo Spring.

Container ambiente que registra Beans, resolve dependências e administra seu ciclo de vida.





A dependência é recebida pronta, em vez de ser criada com `new`.

```
@Service
public class ProdutoService {
    private final ProdutoRepository repository;

    public ProdutoService(ProdutoRepository repository) {
        this.repository = repository;
    }
}
```

- Explicita dependências obrigatórias.
- Facilita testes e permite campos `final`.



@Autowired solicita que o Spring resolva uma dependência automaticamente.

- Pode ser aplicado a construtores, métodos e campos.
- Em uma classe com **um único construtor**, a anotação no construtor é dispensável.
- Injeção por construtor costuma tornar o projeto mais explícito e testável.



```
@Service
public class ProdutoService {
    // regras de negocio e orquestracao
}

@Repository
public interface ProdutoRepository
    extends JpaRepository<Produto, Long> {
    // acesso e consultas ao banco de dados
}
```

Ambos registram componentes no container, mas comunicam responsabilidades distintas da arquitetura.



Use-os quando a criação do objeto precisa ser declarada explicitamente.

```
@Configuration
public class AppConfig {

    @Bean
    public ObjectMapper objectMapper() {
        return new ObjectMapper();
    }
}
```

`@Configuration` classe que fornece definições de Beans.

`@Bean` método cujo retorno será gerenciado pelo container.

Jakarta, persistência e validação



Jakarta EE é um conjunto de especificações padronizadas para aplicações Java empresariais.

- `jakarta.persistence`: mapeamento objeto–relacional.
- `jakarta.validation`: regras declarativas de validação.

O Spring integra essas especificações; elas não são a mesma coisa que Spring Boot.



```
@Entity
public class Produto {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String nome;
    private BigDecimal preco;
}
```

`@Entity` classe persistente mapeada para o modelo relacional.

`@Id` identifica a chave primária.

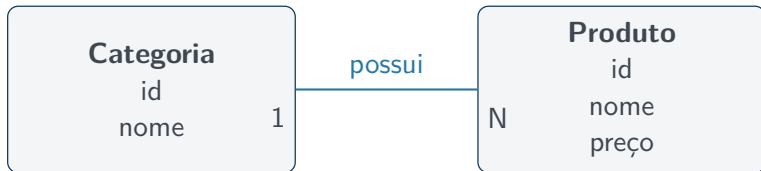
`@GeneratedValue` delega a geração do identificador conforme a estratégia.



Objetos podem referenciar outros objetos. A cardinalidade informa quantas instâncias participam da associação.

Cardinalidade	Anotação	Exemplo
Um para um	@OneToOne	Usuário–Perfil
Muitos para um	@ManyToOne	Produtos–Categoria
Um para muitos	@OneToMany	Categoria–Produtos
Muitos para muitos	@ManyToMany	Produtos–Tags

A cardinalidade deve representar a regra do domínio, não apenas a estrutura do banco.



Uma categoria possui vários produtos; cada produto pertence a uma categoria.



```
@Entity
public class Produto {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String nome;
    private BigDecimal preco;

    @ManyToOne(fetch = FetchType.LAZY, optional = false)
    @JoinColumn(name = "categoria_id", nullable = false)
    private Categoria categoria;
}
```

A tabela de produtos recebe a chave estrangeira categoria_id; por isso, Produto é o lado proprietário.



```
@Entity
public class Categoria {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String nome;

    @OneToMany(mappedBy = "categoria")
    private List<Produto> produtos = new ArrayList<>();
}
```

`mappedBy = "categoria"` aponta para o atributo que controla a associação em `Produto`. Ele evita uma tabela de junção desnecessária.



Em uma associação bidirecional, os dois objetos enxergam o relacionamento, mas apenas um controla seu mapeamento.

Proprietário possui a chave estrangeira ou define a tabela de junção.

Inverso usa `mappedBy` para indicar o atributo proprietário.

Na aplicação

Altere o lado proprietário ao criar a relação: `produto.setCategoria(categoria).`



```
@Service
public class ProdutoService {
    private final ProdutoRepository produtos;
    private final CategoriaRepository categorias;

    public Produto criar(ProdutoRequest request) {
        Categoria categoria = categorias.findById(request.categoriaId())
            .orElseThrow(() -> new CategoriaNaoEncontradaException());

        Produto produto = new Produto();
        produto.setNome(request.nome());
        produto.setPreco(request.preco());
        produto.setCategoria(categoria);

        return produtos.save(produto);
    }
}
```



```
public record ProdutoRequest(  
    @NotBlank String nome,  
    @Positive BigDecimal preco,  
    @NotNull Long categoriaId  
) {}  
  
@PostMapping  
public ResponseEntity<ProdutoResponse> criar(  
    @Valid @RequestBody ProdutoRequest request) {  
    Produto produto = service.criar(request);  
    return ResponseEntity.status(HttpStatus.CREATED)  
        .body(ProdutoResponse.from(produto));  
}
```

O cliente informa o ID; o service busca a entidade relacionada antes de salvar o produto.



```
// Um usuario possui um perfil
@OneToOne(cascade = CascadeType.ALL)
@JoinColumn(name = "perfil_id")
private Perfil perfil;

// Um produto possui varias tags; uma tag pertence a varios produtos
@ManyToMany
@JoinTable(name = "produto_tag",
    joinColumns = @JoinColumn(name = "produto_id"),
    inverseJoinColumns = @JoinColumn(name = "tag_id"))
private Set<Tag> tags = new HashSet<>();
```

@ManyToMany normalmente utiliza uma tabela intermediária com as duas chaves estrangeiras.



- Use DTOs para controlar o JSON e evitar recursão em relações bidirecionais.
- Escolha cascade conforme o ciclo de vida do domínio; não aplique ALL automaticamente.
- Coleções costumam ser carregadas sob demanda; acesse-as dentro do contexto adequado.
- Mantenha os dois lados do objeto sincronizados quando a relação for bidirecional.

Pergunta de projeto

As duas direções são realmente necessárias ou uma associação unidirecional é suficiente?



Anotação	Regra
@NotBlank	texto não nulo, vazio ou só com espaços
@Size	tamanho dentro de um intervalo
@Min	valor maior ou igual ao mínimo
@Max	valor menor ou igual ao máximo

As restrições ficam próximas dos dados que protegem.



```
public record ProdutoRequest(  
    @NotBlank  
    @Size(min = 3, max = 80)  
    String nome,  
  
    @Min(1)  
    @Max(10000)  
    int quantidade  
) {}  
  
@PostMapping  
public Produto criar(@Valid @RequestBody ProdutoRequest request) {  
    return service.criar(request);  
}
```

@Valid solicita a avaliação das restrições do objeto recebido.

Tratamento de erros



Associa um método ao tratamento de um ou mais tipos de exceção.

- Converte falhas da aplicação em respostas HTTP coerentes.
- Pode ser declarado em um controller ou em um componente de aconselhamento global.

Objetivo

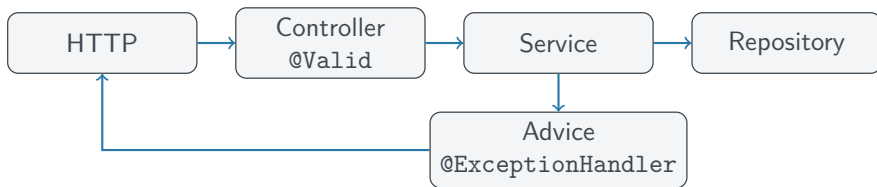
Não expor detalhes internos e oferecer um contrato de erro previsível ao cliente.



Centraliza o tratamento de exceções para vários controllers.

```
@RestControllerAdvice
public class ApiExceptionHandler {

    @ExceptionHandler(ProdutoNaoEncontradoException.class)
    public ResponseEntity<ErroResponse> tratarNaoEncontrado(
        ProdutoNaoEncontradoException ex) {
        return ResponseEntity.status(HttpStatus.NOT_FOUND)
            .body(new ErroResponse(ex.getMessage()));
    }
}
```



Fechamento



- Spring Boot inicializa e autoconfigura a aplicação.
- Controllers mapeiam HTTP; services concentram regras; repositories acessam dados.
- O container aplica IoC e conecta Beans por injeção de dependência.
- Jakarta Persistence mapeia entidades e Jakarta Validation protege entradas.
- Advice e handlers padronizam os erros da API.



Projete um endpoint POST /alunos que:

- 1 Receba nome e idade em JSON.
- 2 Valide nome não vazio e idade entre 16 e 120.
- 3 Delegue a regra de cadastro a um service.
- 4 Persista uma entidade por meio de um repository.
- 5 Retorne erro padronizado quando os dados forem inválidos.

Quais anotações aparecem em cada camada?

Obrigado!

Prof. Dr. João Choma

github.com/JoaoChoma